

UPWORK PROPOSAL

ASSISTANT

Comprehensive Database System Design Document

Technical Specification Manual — Version 2.0



Project Development Group Members:

Sr. No	Group Member Name	Roll Number / ID
1	Tayyab Shahzad	NUM-BSCS-2024-77
2	Najeeba	NUM-BSCS-2024-61
3	Kashif Ali	NUM-BSCS-2024-29

Department of Computer Sciences
Namal University
Mianwali, Pakistan

Document Submission Date: 21st June, 2026

Revision History

Date	Version	Description	Approved by
21/06/2026	V 2.0	Implemented normalized 3NF schema, generated matching DDL/DML scripts with 20+ records, and added specialized admin profile structures.	Asiya Batool
15/05/2026	V 1.0	Final review completed. All conceptual sections proofread and Milestone 2 report submitted.	Asiya Batool
10/05/2026	V 0.3	Business rules drafted and EERD finalized. Relationships and cardinalities verified.	Tayyab Shahzad

Contents

Revision History	1
1 Conceptual Database Design	5
1.1 Entity & Data Dictionary Summary	5
1.2 Data Dictionary Definitions	5
1.2.1 1. Person Table	5
1.2.2 2. Freelancer Table	5
1.2.3 3. Client Table	6
1.2.4 4. Admin Table	7
1.2.5 5. Skills Table	7
1.2.6 6. Freelancer Skill Map Table	7
1.2.7 7. Job Posting Table	8
1.2.8 8. Job Skill Map Table	8
1.2.9 9. Proposal Template Table	8
1.2.10 10. Reliability Score Table	9
1.2.11 11. Freelancer Analytics Table	9
1.2.12 12. Generated Proposals Table	9
2 Logical Database Design	11
2.1 Relational Schema Mapping Rules	11
2.2 Functional Dependencies & Normalization Analysis	11
3 Implementation	13
3.1 Database Engine and Environment Settings	13
3.2 Database Programmability Objects	13
3.3 Interface Payload Mapping	13

List of Tables

1.1	Person Table Schema Definition	6
1.2	Freelancer Table Schema Definition	6
1.3	Client Table Schema Definition	6
1.4	Admin Table Schema Definition	7
1.5	Skills Table Schema Definition	7
1.6	Freelancer Skill Map Table Schema Definition	7
1.7	Job Posting Table Schema Definition	8
1.8	Job Skill Map Table Schema Definition	8
1.9	Proposal Template Table Schema Definition	9
1.10	Reliability Score Table Schema Definition	9
1.11	Freelancer Analytics Table Schema Definition	10
1.12	Generated Proposals Table Schema Definition	10
3.1	Database Programmability Objects Directory	14

Chapter 1

Conceptual Database Design

1.1 Entity & Data Dictionary Summary

Our system architecture rests on a clean, disjoint specialization hierarchy mapping Person into specialized sub-profiles: Freelancer, Client, and Admin[cite: 73]. Supporting tables include Skills, Proposal Template, Job Posting, Reliability Score, and Freelancer Analytics[cite: 74]. All attributes are bound by strict scalar types, primary keys, and mandatory unique index constraints[cite: 75].

1.2 Data Dictionary Definitions

The physical domain attributes, storage allocations, primary keys (PK), foreign keys (FK), and structural constraint configurations for all 12 relations in the Upwork Proposal Assistant database schema are systematically cataloged in the tables below[cite: 78]:

1.2.1 1. Person Table

The Person table tracks foundational data configurations common to all platform profiles[cite: 79].

1.2.2 2. Freelancer Table

The Freelancer entity defines variables specific to independent professional accounts[cite: 81].

Table 1.1: Person Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
person_id	INT	PK	NOT NULL	Auto-incremented identifier.
f_name	VARCHAR(50)	—	NOT NULL	User’s first name.
Lname	VARCHAR(50)	—	NOT NULL	User’s last name.
email	VARCHAR(100)	—	UNIQUE	Primary communication address.
country	VARCHAR(50)	—	NULL	Geographic registration location.
upwork_profile_url	VARCHAR(255)	—	UNIQUE	Direct external link validation.

Table 1.2: Freelancer Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
freelancer_id	INT	PK, FK	NOT NULL	References Person(person_id).
password	VARCHAR(255)	—	NOT NULL	Hashed user authentication string.
hourly_rate	DECIMAL(6,2)	—	NOT NULL	Service fee rate (≥ 0.00).
experience_years	INT	—	NULL	Total industry tenure length.

1.2.3 3. Client Table

The Client relation records operational data profiles for active project employers[cite: 83].

Table 1.3: Client Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
client_id	INT	PK, FK	NOT NULL	References Person(person_id).
company_name	VARCHAR(100)	—	NULL	Registered enterprise title.
success_rate	DECIMAL(5,2)	—	NULL	Historical hiring balance ratio.
feedback	TEXT	—	NULL	Qualitative aggregate reviews.

1.2.4 4. Admin Table

The Admin framework outlines governance credentials across platform elements[cite: 86].

Table 1.4: Admin Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
admin_id	INT	PK, FK	NOT NULL	References Person(person_id).
role	VARCHAR(50)	—	NOT NULL	Admin responsibility designation.
joined_date	DATE	—	NOT NULL	Date of administrative assignment.
status	VARCHAR(20)	—	NOT NULL	Active / Inactive states tracker.

1.2.5 5. Skills Table

The Skills table acts as a master directory cataloging technical competencies[cite: 88].

Table 1.5: Skills Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
skill_id	INT	PK	NOT NULL	Unique index lookup key.
skill_name	VARCHAR(100)	—	UNIQUE	e.g., 'MySQL', 'LaTeX', 'Python'.
skill_level	VARCHAR(30)	—	NOT NULL	Beginner, Intermediate, Expert.

1.2.6 6. Freelancer Skill Map Table

This associative entity defines structural mapping configurations between users and competencies[cite: 90].

Table 1.6: Freelancer Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
freelancer_id	INT	PK, FK	NOT NULL	References Freelancer(freelancer_id).
skill_id	INT	PK, FK	NOT NULL	References Skills(skill_id).

1.2.7 7. Job Posting Table

The Job_Posting table catalogs structural details for individual contract listings[cite: 93].

Table 1.7: Job Posting Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
job_id	INT	PK	NOT NULL	Auto-incremented tracking key.
client_id	INT	FK	NOT NULL	References Client(client_id).
job_title	VARCHAR(150)	—	NOT NULL	Title of the marketplace listing.
job_description	TEXT	—	NOT NULL	Technical scope requirements text.
experience_level	VARCHAR(20)	—	NOT NULL	Bound check: Entry, Mid, Expert.
job_type	VARCHAR(20)	—	NOT NULL	Fixed-price or Hourly configuration.
project_duration	VARCHAR(50)	—	NULL	Expected assignment window length.
budget_amount	DECIMAL(10,2)	—	NOT NULL	Total maximum allocated funding.
posted_date	TIMESTAMP	—	NOT NULL	System generated creation log timestamp.
proposal_count	INT	—	DEFAULT 0	Tracks total bid attempts applied.

1.2.8 8. Job Skill Map Table

Maps the explicit skills necessary for technical evaluation criteria on contracts[cite: 95].

Table 1.8: Job Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
job_id	INT	PK, FK	NOT NULL	References Job_Posting(job_id).
skill_id	INT	PK, FK	NOT NULL	References Skills(skill_id).

1.2.9 9. Proposal Template Table

The Proposal_Template repository tracks standard template data architectures[cite: 98].

Table 1.9: Proposal Template Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
template_id	INT	PK	NOT NULL	Unique structure tracking layout key.
admin_id	INT	FK	NULL	References Admin(admin_id).
template_name	VARCHAR(100)	—	NOT NULL	Title label (e.g., 'React Basic').
category	VARCHAR(50)	—	NOT NULL	Domain classification meta-data tag.
template_content	TEXT	—	NOT NULL	The boilerplate text structure block.
usage_count	INT	—	DEFAULT 0	Times selected by freelancers.
success_rate	DECIMAL(5,2)	—	NULL	Historical effectiveness metrics conversion.

1.2.10 10. Reliability Score Table

The Reliability_Score entity logs metric trends for risk profiling[cite: 100].

Table 1.10: Reliability Score Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
score_id	INT	PK	NOT NULL	Metric entry snapshot record identity.
client_id	INT	FK	NOT NULL	References Client(client_id).
score	DECIMAL(5,2)	—	NOT NULL	Computed trust balance tally index.
risk_level	VARCHAR(20)	—	NOT NULL	System flag: Low, Medium, High.
recommendation_level	VARCHAR(50)	—	NOT NULL	Evaluation: Highly Recommended, Avoid.
calculated_date	TIMESTAMP	—	NOT NULL	Recalculation update execution time.

1.2.11 11. Freelancer Analytics Table

Maintains execution logs summarizing system user conversion parameters[cite: 103].

1.2.12 12. Generated Proposals Table

Tracks explicit transactional logs produced through dynamic cover letter creation[cite: 105].

Table 1.11: Freelancer Analytics Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
analytics_id	INT	PK	NOT NULL	Analytics execution capture token row.
freelancer_id	INT	FK	NOT NULL	References Freelancer(freelancer_id).
total_proposals	INT	—	DEFAULT 0	Total job submissions tracking counter.
accepted_proposals	INT	—	DEFAULT 0	Total accepted hire items registry.
rejected_proposals	INT	—	DEFAULT 0	Total closed/passed applications logs.
success_rate	DECIMAL(5,2)	—	DEFAULT 0.00	Percentage representation profile conversion.

Table 1.12: Generated Proposals Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Description / Constraints
proposal_id	INT	PK	NOT NULL	Transaction logging instance primary token.
freelancer_id	INT	FK	NOT NULL	References Freelancer(freelancer_id).
client_id	INT	FK	NOT NULL	References Client(client_id).
job_id	INT	FK	NOT NULL	References Job_Posting(job_id).
template_id	INT	FK	NOT NULL	References Proposal_Template(template_id).
proposal_content	TEXT	—	NOT NULL	Customized generated final cover letter.
generated_date	TIMESTAMP	—	NOT NULL	Time system algorithm structured row.
proposal_status	VARCHAR(30)	—	NOT NULL	Pending, Submitted, Accepted, Rejected.
submission_date	TIMESTAMP	—	NULL	Time submitted directly to client.

Chapter 2

Logical Database Design

2.1 Relational Schema Mapping Rules

The database tables are structured based on Third Normal Form (3NF) relational criteria to achieve high structural stability[cite: 114]. The blueprint maps the conceptual specialization model into concrete physical relations using a specialized table-per-child hierarchy strategy. Under this arrangement, the parent entity Person maintains identity tokens common to all actors on the platform. Sub-profiles extend these parent definitions. Rather than duplicating common descriptors across multiple endpoints, child relations maintain strict referencing integrity by reusing the parent's identifier as their primary key while binding it concurrently under an index-validated foreign key constraint. Associative intersection bridges decouple many-to-many linkages into explicit key-pairs, ensuring that multi-valued attributes remain atomic.

2.2 Functional Dependencies & Normalization Analysis

All data fields map directly and explicitly to primary composite candidate keys[cite: 117]. The structural layout of all 12 operational relations is thoroughly verified to meet the performance bounds of Third Normal Form (3NF).

1. **First Normal Form (1NF):** Every structural item is strictly atomic. Text and description frameworks avoid packed arrays by isolating repeating sequences.
2. **Second Normal Form (2NF):** All transactional schemas possess full functional dependency on their primary tracking elements. In relations with composite primary configurations, such as mapping bridges, partial dependencies are completely absent.

3. **Third Normal Form (3NF):** Transitive functional dependencies are completely eliminated. Descriptive text fields depend directly on the primary key identifier, avoiding indirect functional dependencies on secondary variables.

Chapter 3

Implementation

3.1 Database Engine and Environment Settings

The data infrastructure runs concurrently under a local instance of the MySQL 8.0 Engine[cite: 123]. The underlying platform maps physical data sectors via the InnoDB transaction processing engine. This setup handles strict enforcement of database ACID properties (Atomicity, Consistency, Isolation, Durability) alongside native row-level write-locking mechanics during active query loads.

3.2 Database Programmability Objects

The programmable logic layer implements backend control mechanisms via stored routines and data tracking automation rules[cite: 125]. Table 3.1 outlines these functional behaviors[cite: 130, 131]:

3.3 Interface Payload Mapping

The system layout registers connection paths securely handling transactional payloads smoothly[cite: 128]. This section serves as a descriptive blueprint detailing how the platform’s backend relational schema links with external operational frontends. The core purpose of the data schemas defined in Chapter 1 is to seamlessly pass data payloads between the database instances and client interfaces without structural formatting bottlenecks or string escape failures.

By standardizing primary identity references across individual staging arrays, the data catalog establishes a structured map that application connectors can query instantly. This foundational layout ensures that when subsequent development phases introduce runtime interface integrations, the pre-built table schemas are fully prepared to support

Table 3.1: Database Programmability Objects Directory

Sr.	Object Name	Object Type	Description	Input / Output Parameters
01	Calculate_Reliability	Stored Procedure	Evaluates client success rates against policy data, updates risks, and appends a score row.	Input: c_id INT, raw_score DECIMAL Output: Inserts to Reliability_Score.
02	Increment_Template_Usage	Trigger	Automatically increments the template usage counter when a new proposal row is added.	Event: AFTER INSERT on Generated_Proposals table.
03	Sync_Freelancer_Metrics	Stored Procedure	Automatically recalculates totals, acceptance counters, and success rate values for a freelancer.	Input: f_id INT Output: Modifies row inside Freelancer_Analytics.

stable, continuous system querying.

Bibliography

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016[cite: 133, 134].
- [2] MySQL Reference Manual, “Stored Procedures, Functions, and Triggers Infrastructure,” Oracle Corp, 2026[cite: 134].

Appendix: AI Usage and Prompts

In compliance with project design evaluation guidelines, the development prompts used with generative artificial intelligence models are documented below[cite: 135]:

- **Prompt 1 (Schema Refactoring):** “Review my Upwork database design layout attributes. Fix any potential decimal data type truncation bottlenecks within the job posting financial budgets and client reliability performance score range values to ensure it safely handles up to 100% metric values without runtime crashes.” [cite: 135]
- **Prompt 2 (Transactional Scaling):** “Generate a fully expanded, interconnected relational seed script mapping out 2 distinct administrative roles (one for system user metrics analysis, one for outgoing template tracking) along with more than 20 rows of realistic transactional mock records per operational table to test structural performance.” [cite: 136]
- **Prompt 3 (Documentation Mapping):** “Map my updated relational set structures into a comprehensive logical validation report structured under formal Chapter 3 and Chapter 4 sections following Namal University project templates.” [cite: 137]